

MINI PROJECT WORK

(PAD21P02L)

On

TACTICAL FORMATION DETECTION IN FOOTBALL USING GNN+LSTM

By

ALBIN MICHAEL

RA2432014020008

Submitted to

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS (M.Sc. Applied Data Science)

Under the guidance of

Dr. R. RENUGA DEVI

(Associate Professor)

MASTER OF SCIENCE – APPLIED DATA SCIENCE



SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Ramapuram Campus, Chennai.

NOVEMBER 2025



FACULTY OF SCIENCE AND HUMANTIES

(A Place for transformation)

Ramapuram, Chennai.



Department of Computer Science & Applications

(M.Sc. Applied Data Science)

BONAFIDE CERTIFICATE

Certified that this project report titled as **“Tactical Formation Detection in Football using GGN+LSTM”** is the bonafide work of **ALBIN MICHAEL (RA2432014020008)** who carried out the “PAD21P02L – Mini Project Work” done under my supervision.

Certified further, that to the best of my knowledge the work reported here in does not form any part of any project report on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Signature of Internal Guide

Signature of Head of the Department

Signature of External Examine

ACKNOWLEDGEMENT

I extend my sincere gratitude to the Chancellor **Dr. T.R. PACHAMUTHU** and to Chairman **Dr. R.SHIVAKUMAR** of SRM Institute of Science and Technology, Ramapuram campus for providing me the opportunity to pursue the M.Sc. (Applied Data Science) degree at this University.

I express my sincere gratitude to **Dr. S.THIRUMAGAN, Dean(S&H)**, and SRMIST, Ramapuram for his support and encouragement for the successful completion of the project.

I record my sincere thanks to **Dr. J. DHILIPAN Professor & Vice – Principal (Admin-S&H)**, and **Dr.N.PUGHAZENDI Professor & Vice – Principal (Academic-S&H)**, SRMIST, Ramapuram for his continuous support and keen interest to make this project a successful one.

I record my sincere thanks to **Dr. N.VIJAYALAKSHMI, Associate Professor**, and Head, Department of Computer Science and Applications, M.Sc. Applied Data Science, SRMIST, Ramapuram for his continuous support and keen interest to make this project a successful one.

I am grateful to my Guide **Dr R. RENUGA DEVI, Associate Professor** Department of Computer Science & Applications, M.Sc. Applied Data Science SRMIST, Ramapuram for his continuous support and keen interest to make this project a successful one.

I thank my almighty who has made this possible. Finally I thank my beloved family members and friends for their motivation, encouragement and cooperation in all aspect which led me to the completion of this project.

ALBIN MICHAEL (RA2432014020008)

ABSTRACT

ABSTRACT

Formation detection in football is crucial for understanding team tactics and strategies, yet it remains a challenging task due to the dynamic nature of player movements and the complexity of spatial-temporal relationships on the field. This study addresses this problem by proposing a novel approach that combines Graph Neural Networks (GNN) with Long Short-Term Memory (LSTM) networks to accurately detect and classify team formations in real-time. The GNN+LSTM approach leverages the spatial relationships between players, represented as nodes in a graph, while capturing the temporal evolution of these relationships throughout the match. The GNN component processes the spatial features of player positions, while the LSTM network models the sequential nature of the game, allowing for more accurate predictions of formation changes over time. This model achieves an overall accuracy of 92% in detecting common football formations, outperforming traditional machine learning methods by a significant margin. The F1-score for individual formation classifications ranges from 0.88 to 0.95, demonstrating robust performance across various tactical setups. These results have significant implications for sports analytics and tactical analysis. The ability to automatically detect formations in real-time can provide coaches and analysts with valuable insights for in-game decision-making, opposition analysis, and post-match review. Furthermore, this approach opens up new possibilities for advanced performance metrics and strategic planning in professional football.

TABLE OF CONTENTS

S. NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	i
	ABSTRACT	ii
CHAPTERS	TITLE	PAGE NO.
1	INTRODUCTION 1.1 PROJECT DESCRIPTION	1
2	SYSTEM ANALYSIS 2.1 SOURCE OF DATASET 2.2 FEASIBILITY STUDY 2.3 EXISTING SYSTEM AND DRAWBACKS 2.4 PROPOSED SYSTEM 2.5 BENEFITS OF PROPOSED SYSTEM 2.6 SCOPE OF PROJECT 2.7 OUTPUT ENVIRONMENT	5
3	SYSTEM SPECIFICATION 3.1 HARDWARE CONFIGURATION 3.2 SOFTWARE CONFIGURATION	17
4	SYSTEM DESIGN 4.1 SOFTWARE MODULE 4.2 ARCHITECTURE	20

5	PROJECT DESCRIPTION 5.1 MODULE DESCRIPTION 5.2 SYSTEM MAINTENANCE	25
6	TESTING 6.1 DATA AND PREPROCESSING VALIDATION 6.2 CORE MODEL PERFORMANCE 6.3 ROBUST AND DEPLOYMENT TESTING	30
7	CONCLUSION 7.1 PROJECT SUMMARY 7.2 FUTURE ENHANCEMENT	36
8	APPENDIX 8.1 SYSTEM DESIGN 8.2 ARCHITECTURE DIAGRAM 8.3 CONFUSION MATRIX AND MODEL METRICS COMPARISION 8.4 SCREENSHOTS 8.5 CODING	43
9	BIBLIOGRAPHY	63

CHAPTER 1

INTRODUCTION

INTRODUCTION

1.1 PROJECT DESCRIPTION

In modern football, understanding team formations and tactical structures has become a key component of performance analysis and strategic planning. A team’s formation — the spatial arrangement and movement of players across the pitch — determines how effectively it can attack, defend, and transition between phases of play. Traditionally, these formations were analyzed manually by coaches or video analysts, which is a time-consuming and subjective process. The rise of **player tracking technologies** and **data analytics** has transformed this field, allowing precise, frame-by-frame recording of player positions, speeds, and accelerations throughout a match. However, extracting meaningful tactical insights from such high-dimensional, spatio-temporal data remains a significant challenge.

Recent advancements in **Artificial Intelligence (AI)**, particularly in **Deep Learning**, have opened new avenues for automated tactical analysis. Among these, **Graph Neural Networks (GNNs)** have shown strong potential for modeling interactions among entities — in this case, players — represented as nodes connected by spatial or relational edges. When combined with **Recurrent Neural Networks (RNNs)** such as **Long Short-Term Memory (LSTM)** models, this architecture can effectively capture both **spatial dependencies** (how players interact and maintain formation) and **temporal dynamics** (how these formations evolve over time).

This project proposes a **hybrid GNN–LSTM framework** for detecting and analyzing tactical formations in football using player **tracking** and **event data**. The system reads positional (x, y) and kinematic (velocity, acceleration, speed) data for all players on the pitch and constructs a **graph representation** of each frame, where nodes represent players and edges connect spatially proximate players. The **GNN layer** captures the relationships and interaction patterns among teammates, while the **LSTM layer** models the evolution of these spatial structures over time. This enables the system to recognize consistent tactical shapes and formation transitions (e.g., 4-3-3 to 4-2-3-1) without manual labeling.

To enhance the reliability of formation recognition, the model also uses **event data filtering** to exclude non-active periods such as stoppages and halftime intervals. The resulting data focuses purely on active play segments, improving clustering accuracy. The detected

formations are then grouped using **K-Means clustering**, and evaluated using **Accuracy**, **F1-score**, and **Silhouette score** to measure both classification and cluster quality.

The proposed framework was tested on open football datasets (Metrica Sports tracking data) containing multiple matches and teams. Experimental results demonstrate that the **GNN+LSTM hybrid model** achieved superior performance compared to the baseline **LSTM-only** approach — showing a **45% F1-macro score** and **0.517 accuracy**, compared to **0.183 F1** and **0.357 accuracy** for the baseline. The inclusion of the GNN layer significantly enhanced spatial feature learning by modeling player proximity and interaction.

The system provides a foundation for **automated tactical analysis**, enabling analysts and coaches to quantitatively study how formations adapt during matches, identify defensive gaps, and evaluate positional discipline. Furthermore, this model can be extended to real-time analytics, providing dynamic tactical visualizations and strategy recommendations during live matches.

In summary, this project bridges the gap between **sports analytics** and **deep graph learning**, presenting an interpretable and scalable solution for **formation detection, tactical monitoring, and strategy evaluation** in football. The integration of GNNs for spatial modeling and LSTMs for temporal learning represents a powerful step toward data-driven sports intelligence.

1.2 OBJECTIVE OF THE STUDY

1. A GNN–LSTM hybrid architecture for formation detection that captures both spatial and temporal dependencies.
2. Integration of event-based active play filtering to ensure accurate formation recognition.
3. Exploration of unsupervised formation clustering using K-Means with cluster tuning (n=4–6).
4. Comparative analysis showing significant accuracy and F1 improvements over traditional LSTM baselines.
5. Development of a reproducible pipeline suitable for real-world sports analytics and academic research.

CHAPTER 2

SYSTEM ANALYSIS

SYSTEM ANALYSIS

2.1 SOURCE OF DATASET

The dataset utilized in this research originates from Metrica Sports, a globally recognized provider of open-source football analytics data. The Metrica Sports dataset offers one of the most comprehensive and publicly accessible collections of match tracking and event data used in sports analytics research and development. It has been widely employed in academic and industrial studies for evaluating algorithms in player tracking, tactical formation analysis, and team behavior modeling.

The dataset comprises two key components—tracking data and event data. The tracking data contain precise spatial coordinates (x, y) of every player and the ball at each frame of the match, recorded at high frequency (typically 25 frames per second). Each frame captures the instantaneous position of players from both the home and away teams, providing a dynamic sequence of movements that can be analyzed to infer formations, spacing, and collective team structures. The event data, on the other hand, record significant occurrences during the match—such as passes, shots, tackles, and fouls—along with the start and end frame indices of each event. This dual structure allows researchers to synchronize tactical formations with contextual match situations, offering a more holistic understanding of the game's flow.

In this project, multiple CSV files from the Metrica Sports open dataset were employed, including *tracking_home.csv*, *Sample_Game_1_RawTrackingData_Away_Team.csv*, *Sample_Game_2_RawTrackingData_Home_Team.csv*, and *Sample_Game_2_RawEventsData.csv*. These files collectively cover two full matches and represent both teams' tracking information along with the corresponding event sequences. The data are standardized to maintain consistency in coordinate orientation, ensuring that the home team always attacks from left to right, which simplifies comparative formation analysis.

The Metrica Sports open dataset is available publicly through their official GitHub repository: <https://github.com/metrica-sports/sample-data>.

This open-access nature makes it ideal for academic experimentation, reproducibility, and benchmarking of new analytical models.

To prepare the dataset for model training and evaluation, several preprocessing steps were applied, including:

- Filtering of inactive periods (halftime and stoppages) using event timeframes.
- Calculation of derived kinematic features such as velocity, acceleration, and speed.
- Transformation of the wide-format tracking files into a long format suitable for machine learning input.
- Synchronization of tracking frames with event intervals to focus solely on active play segments.

By utilizing this well-structured and standardized dataset, the project ensures a high level of data integrity, reliability, and comparability with existing literature in football analytics. Furthermore, the open nature of Metrica Sports data encourages continued experimentation, allowing future researchers to extend and validate the models presented in this work.

2.2 FEASIBILITY STUDY

A **feasibility study** evaluates the practicality, resource requirements, and viability of implementing the proposed **GNN + LSTM-based spatio-temporal framework** for football formation detection. The purpose of this study is to determine whether the project can be successfully developed and deployed using available technologies, datasets, and computational resources within the defined constraints of time and cost.

1. Technical Feasibility:

The proposed system is technically feasible, as it is built upon existing and well-established technologies in data science and deep learning. The framework integrates **Graph Neural Networks (GNN)** and **Long Short-Term Memory (LSTM)** architectures to model player interactions and temporal patterns in football tracking data.

a. Data Compatibility

The **Metrica Sports Open Data** and similar datasets provide **player tracking** and **event logs**

in structured CSV format, which are compatible with the framework. These datasets include x–y coordinates for each player at every frame and event metadata such as ball possession and stoppages, enabling preprocessing and filtering of active play segments

b. Technical Viability

- The combination of **GNN (for spatial structure learning)** and **LSTM (for temporal sequence modeling)** has been proven effective in several spatio-temporal domains (traffic flow, human activity recognition, etc.), ensuring adaptability to football analytics.
- The project design supports **scalability** — additional matches or players can be processed without redesigning the architecture.
- With PyTorch’s GPU acceleration and modular design, the system can handle thousands of frames efficiently.
- The modular pipeline allows future upgrades, such as integrating **real-time streaming data**, **transformer-based attention models**, or **reinforcement learning** for tactical decision-making.

2. Economic Feasibility

The project is **cost-effective** because it relies exclusively on **open-source tools and publicly available data**. No proprietary software or commercial licensing is required.

a. Cost Components

- **Development Cost:** Limited to computational resources (electricity, GPU time, and system maintenance).
- **Data Acquisition:** Metrick Sports datasets and similar open datasets are freely available.
- **Software Licensing:** All frameworks (PyTorch, scikit-learn, pandas, etc.) are open-source under permissive licenses (MIT/BSD).
- **Personnel Cost:** Single-developer or academic project team effort — no external labor costs.

b. Cost vs. Benefit

- **Short-term benefits:** A functional prototype capable of detecting tactical formations and evaluating team dynamics.
- **Long-term benefits:** The methodology can be commercialized or integrated into sports analytics platforms for real-time tactical insights.
- **Return on Investment (ROI):** High potential ROI in research and sports analytics contexts with minimal financial expenditure.

3. Operational Feasibility

Operational feasibility determines whether the system can be effectively used and maintained once deployed.

a. User Accessibility

- The system is designed to be **analyst-friendly**, with outputs in the form of cluster visualizations, accuracy tables, and formation comparison graphs.
- The user interface can be extended into a simple dashboard for analysts and coaches to visualize formation shifts and team dynamics.
- Minimal user intervention is required after setup, as data preprocessing, clustering, and model training are automated.

b. Reliability and Maintainability

- The pipeline is modular — components such as **data preprocessing**, **model training**, and **visualization** are decoupled for ease of debugging and updates.
- Model parameters and results are reproducible due to fixed random seeds and standardized data input.
- Updates in model architecture (e.g., new GNN layers or longer LSTM horizons) can be integrated seamlessly.

c. Integration and Usability

- The system can be integrated with sports video annotation tools or match visualization platforms.
- Possible extensions include compatibility with APIs for **real-time tracking systems** (e.g., GPS or optical tracking).

5. Legal and Ethical Feasibility

- The datasets used are **publicly released for research** and educational purposes under open licenses (e.g., Metrica Sports Open Data).
- No personally identifiable information (PII) is included; data pertains to anonymous player IDs and in-game positions.
- The research adheres to **ethical AI practices**, focusing on performance improvement and data transparency without misuse of athlete data.
- Proper **citations and acknowledgments** are included for all datasets and prior research works (IEEE/Elsevier standards).

2.3 EXISTING SYSTEM AND DRAWBACKS

The analysis of tactical formations and player positioning in football has traditionally relied on **manual observation**, **static statistical metrics**, and **simple clustering algorithms**. These methods provide limited insights into the dynamic and complex nature of team movement, coordination, and formation transitions that occur throughout a match.

Existing systems can broadly be divided into three categories: **manual and video-based systems**, **statistical and geometric analysis methods**, and **machine learning-based clustering approaches**.

1. Manual and Video-Based Analysis

In the early stages of football analytics, tactical analysis was performed manually by **coaches, scouts, and analysts** reviewing match footage. Using frame-by-frame video playback, analysts identified patterns such as player spacing, pressing structures, and off-ball movement.

While this approach provides **contextual understanding**, it suffers from several limitations:

- It is **subjective**, as insights depend on the analyst's experience and perception.
- It is **time-consuming**, requiring extensive manual review of game footage.
- The analysis is typically **static**, focusing on specific moments rather than continuous, time-dependent tactical evolution.
- Quantitative metrics, such as inter-player distance or shape compactness, are rarely computed, making objective comparison between teams difficult.

2. Statistical and Geometric Tracking Approaches

With the advent of **optical tracking and GPS-based data**, research shifted toward quantitative analysis of formations. In these systems, player coordinates are extracted at each time frame and used to compute basic geometric measures:

- **Team centroid and dispersion** (average player positions and spread)
- **Convex hull area** of player positions to measure compactness

- **Inter-player distances** and **positional heatmaps**

For example, the work of **Bialkowski et al. (2014, 2016)** introduced spatial modeling using tracking data to quantify team shape and identify formations through positional averages. These approaches used statistical methods like **Principal Component Analysis (PCA)** and **K-Means clustering** to classify formations (e.g., 4-3-3, 4-4-2) from average coordinates.

However, such methods make several **simplifying assumptions**:

- They assume that player positions are **independent**, ignoring inter-player influence and interactions.
- They treat each frame or average position **in isolation**, without modeling temporal continuity (i.e., how formations evolve over time).
- They rely heavily on **predefined features**, such as distance and area metrics, rather than learning representations directly from data.

Consequently, these methods are limited in identifying **dynamic transitions** (e.g., attacking vs. defensive shape changes) and **contextual variations** (e.g., player rotations or substitutions).

3. Machine Learning and Clustering-Based Formation Detection

More recent systems incorporate **unsupervised learning** to identify tactical formations automatically.

The **K-Means clustering algorithm** is commonly used to group player positions into formation clusters.

Some studies have used **DBSCAN (Density-Based Spatial Clustering)** for detecting compact groups without predefined cluster counts. These clustering techniques provide a degree of automation but still face major limitations:

- **No temporal modeling:** Each frame is clustered independently, losing continuity between consecutive frames.
- **Sensitivity to outliers:** Player swaps or off-ball runs distort clusters.
- **Static spatial assumptions:** Formations are assumed to be rigid; these methods cannot track fluid tactical transitions.

- **Limited interpretability:** Cluster assignments do not inherently capture tactical roles or inter-player dependencies.

In addition, existing clustering models lack generalization across matches or teams, as the learned cluster centroids are dataset-specific and do not adapt well to new conditions.

4. Limitations of the Existing System

Despite advancements in tracking technology and data-driven analytics, current systems face several technical and analytical challenges:

Aspect	Existing Method	Limitation
Data Handling	Tracking data + manual filters	No dynamic filtering for active play
Spatial Modeling	K-Means, PCA, geometric features	Ignores graph-based inter-player relations
Temporal Modeling	Frame-by-frame or static averages	No sequence-level temporal learning
Adaptability	Hand-crafted features	Poor Generalization to un seen matches
Output Quality	Basic clustering visualization	Lacks interpretable formation evolution

The inability to represent **spatial dependencies (player–player relationships)** and **temporal dependencies (sequence evolution)** limits the performance and reliability of these systems. The need for a **robust, data-driven, and adaptive model** capable of learning from both spatial and temporal dynamics motivates the development of the proposed **GNN + LSTM Spatio-Temporal Framework**.

2.4 PROPOSED SYSTEM

The proposed system introduces a hybrid deep learning framework that integrates Graph Neural Networks (GNNs) with Long Short-Term Memory (LSTM) networks to accurately detect and analyze football team formations from tracking and event data. Unlike traditional clustering or static analysis methods, this model learns both the spatial dependencies among players and the temporal evolution of formations during a match,

enabling a dynamic, data-driven understanding of tactical behavior.

The system processes multi-match datasets containing player positions, velocities, and accelerations, along with event data that indicates periods of active play. It constructs spatio-temporal graphs, where nodes represent players and edges represent contextual interactions (e.g., proximity or passing potential). Through this integrated pipeline, the system performs formation recognition, pattern clustering, and visualization of team dynamics.

1. System Overview

The proposed architecture consists of several computational layers and modules that work together to capture the complexities of football formations:

1. Data Acquisition and Preprocessing

- The system ingests *Metrika Sports open-source tracking data* containing player coordinates (x, y) and velocity information for each frame, as well as *event files* marking active play sequences.
- Data preprocessing involves:
 - Filtering active frames (using event timestamps to exclude stoppages and halftime).
 - Standardizing field orientation so the home team always attacks in the same direction.
 - Feature engineering by computing kinematic features such as velocity, acceleration, and speed for each player.
- Graph construction, where nodes = players and edges = k-nearest neighbors (KNN) based on proximity.

2. Graph Construction and Embedding

- Each frame is converted into a graph structure:
- Nodes: Represent individual players with feature vectors [x, y, vx, vy, speed, ax, ay].
- Edges: Determined using K-Nearest Neighbor (KNN) relationships or team-based proximity to capture inter-player dependencies.
- Graph embedding is performed using a Graph Attention Network (GATConv), which dynamically learns the strength of influence between players based on their interactions.

3. Temporal Modeling with LSTM

- The temporal evolution of formations (over several frames) is captured using a Long Short-Term Memory (LSTM) network.
- The LSTM processes a sequence of per-frame graph embeddings to learn the transition patterns between formations.

- This enables detection of both *static formations* (like 4-3-3 or 3-5-2) and *dynamic transitions* (e.g., attacking or defending phase changes).

4. Formation Clustering and Classification

- After obtaining temporal embeddings, K-Means clustering is applied to detect natural formation groupings.
- The model is tuned to sweep cluster counts ($k = 4, 5, 6$) for optimal formation recognition accuracy.
- The predicted formations are evaluated using metrics such as Accuracy, F1-score, and Silhouette score to ensure both classification reliability and cluster quality.

5. Visualization and Insights

- The system generates formation heatmaps, player position trajectories, and cluster maps that provide visual explanations of tactical structures.
- These visualizations can be used by coaches, analysts, and sports scientists to interpret formation stability and player movements.

2.5 BENEFITS OF PROPOSED SYSTEM

1. Dynamic Formation Detection – Captures time-dependent player interactions using a spatio-temporal deep learning model.
2. Improved Accuracy – Combines GNN-based spatial reasoning with LSTM-based temporal analysis for higher classification precision.
3. Automatic Event Filtering – Integrates event data to remove inactive frames and focus on relevant match phases.
4. Robust and Scalable – Can generalize across teams, matches, and tournament datasets with minimal retraining.
5. Explainable Outputs – Generates interpretable formation heatmaps and cluster visualizations for practical tactical insights.

2.6 SCOPE OF PROJECT

The scope of this project extends to the comprehensive understanding, detection, and visualization of football team formations using advanced machine learning and deep learning techniques. The analysis focuses on leveraging both spatial and temporal aspects of player

movement to gain deeper insights into tactical behaviors exhibited during different phases of play. Unlike traditional methods that rely on manual tagging or simple clustering of static positions, this research explores a data-driven and automated pipeline capable of learning complex interaction patterns directly from raw tracking and event data.

At the foundation of this analysis lies the integration of Graph Neural Networks (GNNs) and Long Short-Term Memory (LSTM) architectures. This hybrid framework is specifically designed to address the multidimensional nature of football tracking data, where every player's position and motion are interdependent and evolve dynamically over time. The GNN component captures spatial dependencies between players by modeling their positional relationships as graph structures, while the LSTM component captures temporal dependencies, learning how formations shift and adapt across consecutive frames. Together, these models enable the system to recognize both stable formations, such as 4-3-3 or 3-5-2, and fluid transitions during attacks or defensive reorganizations.

The scope of this study also encompasses the application of event-driven data filtering to ensure that only active phases of play are considered for analysis. This approach enhances data quality by excluding halftime, stoppages, and other non-play segments that could introduce noise or distort formation learning. Furthermore, the project explores unsupervised clustering techniques, such as K-Means, to automatically identify tactical formations without the need for predefined labels. The analysis is extended across multiple matches using open-source datasets such as those provided by Metrica Sports, enabling the framework to generalize across teams and match conditions.

Beyond algorithmic development, the scope also includes the evaluation and interpretation of the detected formations through visualization tools. These visualizations, including heatmaps, trajectory plots, and cluster maps, provide meaningful insights into player distribution and collective team structure. Such outputs are vital for bridging the gap between computational analysis and practical sports applications, allowing coaches, analysts, and performance scientists to understand and refine tactical decisions.

In a broader sense, the project's scope contributes to the emerging field of sports analytics through artificial intelligence, particularly in the area of tactical intelligence and strategy modeling. It establishes a scalable and extensible analytical framework that can be adapted to other team-based sports, such as basketball or hockey, where formation and movement patterns also play a crucial role. Ultimately, this work aims to transform raw positional data into actionable knowledge, supporting more objective, consistent, and insightful analysis of football tactics through the synergy of graph-based learning and temporal sequence modeling.

CHAPTER 3

SYSTEM SPECIFICATIONS

SYSTEM SPECIFICATION

3.1 HARDWARE CONFIGURATION

The system requires a moderately powerful setup to handle the spatio-temporal tracking data and accelerate the deep learning model training.

Core Components

- Processor (CPU):
 - Minimum: Intel Core i5 or AMD equivalent (quad-core or higher).
 - Purpose: Data preprocessing, general system operation.
- Random Access Memory (RAM):
 - Minimum: 8 GB.
 - Recommended: 16 GB.
 - Purpose: Handling large tracking data frames in memory during training and preprocessing.
- Storage:
 - Type: 512 GB Solid State Drive (SSD) or higher.
 - Benefit: Provides faster data loading and access compared to a traditional HDD.

Acceleration Component

- Graphics Processing Unit (GPU):
 - Minimum: Integrated graphics are sufficient for basic execution.
 - Recommended: A dedicated GPU, specifically an NVIDIA GTX 1050 or higher.
 - Purpose: Accelerating the computationally intensive training of the GNN (Graph Attention Network) and LSTM models.

Operating System

- OS Compatibility: Windows 10/11, macOS, or Linux (e.g., Ubuntu).
- Requirement: Must support the necessary Python environment and machine learning frameworks.

3.2 SOFTWARE CONFIGURATION

A compatible software environment is essential for running the project code.

- Programming Language: Python 3.8+
- Machine Learning Frameworks: PyTorch, NumPy, Pandas, Scikit-learn.
- Graph Neural Network Tool: PyTorch Geometric (Required for GATConv implementation).
- Data Visualization: Matplotlib and Seaborn.

CHAPTER 4

SYSTEM DESIGN

SYSTEM DESIGN

4.1 SOFTWARE MODULE

1. Data Acquisition and Preprocessing Module

This module is responsible for ingesting raw data and preparing it for deep learning and clustering analysis.

- Core Function: Reading raw tracking and event CSV files.
- Active Play Filtering: Filters the tracking data to exclude non-active periods (e.g., stoppages, halftime) by synchronizing frame IDs with event log intervals.
- Orientation Standardization: Adjusts player coordinates (flips x and potentially y values in the second half) to ensure the target team always attacks left-to-right.
- Kinematic Feature Computation: Calculates player kinetics, such as velocity (v_x , v_y), speed, and acceleration (a_x , a_y) for each player at each frame.

2. Graph Construction Module

This module converts the preprocessed positional data into a graph format suitable for the GNN encoder.

- Graph Builder: For every time step/frame, it dynamically constructs a graph where nodes represent individual players (with kinematic features) and edges represent spatial relationships.
- Edge Definition: Typically uses K-Nearest Neighbors (k-NN) based on player proximity to define edges (e.g., $k=4$), capturing immediate spatial context and team cohesion.

3. Deep Learning Framework Module (GNN + LSTM Core)

This module contains the primary spatio-temporal recognition logic, typically built using PyTorch and PyTorch Geometric.

- Feature Extraction (GNN Encoder): Employs a Graph Attention Network (GATConv) layer to process the per-frame graphs. The GNN learns a dense node embedding that captures who is playing where and how their local interactions influence their roles (spatial context).
- Temporal Modeling (LSTM): A Long Short-Term Memory (LSTM) network is used to process the sequence of GNN embeddings (over a defined time window T). This layer specializes in modeling the temporal dynamics of the formation, allowing for the detection of subtle formation transitions and shifts.

4. Clustering and Classification Module

This module provides the final formation label and forms the basis for comparative analysis.

- Clustering (Unsupervised Labeling): Uses K-Means or DBSCAN on the averaged player positions (or output embeddings) across a time window to generate pseudo-labels for distinct formation groups (e.g., grouping the sequences into 4, 5, or 6 distinct tactical classes).
- Supervised Classification: A Softmax layer (or equivalent) takes the final temporal output from the LSTM and predicts the formation cluster/class for the entire sequence.

5. Visualization and Evaluation Module

This module generates interpretable outputs and measures model performance.

- Evaluation Metrics: Computes quantitative metrics for model comparison, including Accuracy, F1-score, and Silhouette score (to assess cluster quality).
- Visual Reports: Generates graphical outputs for human interpretation, such as formation heatmaps, player trajectory maps, cluster diagrams, and pass network graphs.

4.2 ARCHITECTURE

1. The Input Layer: Feature Extraction

The architecture begins by processing the raw, preprocessed tracking data for a defined time window T (the sequence length).

- Input Data: The architecture ingests a sequence of player feature vectors $[(x, y, vx, vy, ax, ay, \dots)]$.
- Time Window (T): The model doesn't look at a single frame but a sequence of T frames (e.g., $T=30$ frames) to understand *how* the formation is moving, not just *where* it is.

2. The Spatial Encoder: Graph Neural Network (GNN)

This module handles the non-Euclidean, relational data (player positions). It transforms the positional frame into a context-aware embedding.

- Graph Construction: At each time step t , the system constructs a graph G_t where:
 - Nodes (V): Represent individual players, using their kinematic feature vectors as node features.
 - Edges (E): Represent spatial proximity, typically defined using k-Nearest Neighbors (k-NN) based on player distance.
- GNN Layer (GATConv): A Graph Attention Network (GATConv) is used as the core GNN. This layer allows the model to:
 - Learn Weighted Relationships: Assign attention scores to edges, giving more importance to

interactions with certain nearby players (e.g., a defender's relationship with the central midfielder is more important than with the furthest striker).

- Output: For each frame t , the GNN outputs a Spatial Embedding Vector (E_t) that aggregates the context-aware features of all players in the formation.

3. The Temporal Encoder: Long Short-Term Memory (LSTM)

This module is responsible for capturing the dynamics and sequential nature of the tactical structure.

- Input Sequence: The LSTM receives the sequence of spatial embedding vectors: $E_{t-T}, E_{t-T+1}, \dots, E_t$.
- LSTM Mechanism: The LSTM's recurrent nature allows it to remember the spatial state of the formation over the entire time window T and forget irrelevant past states. This is crucial for detecting subtle formation transitions (e.g., shifting from a 4-4-2 press to a 4-5-1 defense).
- Output: The LSTM's final hidden state vector ($\mathbf{H}_t$) serves as a rich, condensed Spatio-Temporal Feature that encapsulates the entire tactical sequence.

4. The Output Layer: Classification Head

This final stage maps the complex spatio-temporal feature into a concrete tactical label.

- Input: The final hidden state vector $\mathbf{H}_t$.
- Softmax Classifier: A fully connected (Dense) layer followed by a Softmax activation function is used to classify the feature vector $\mathbf{H}_t$ into one of the pre-defined formation classes (e.g., '4-3-3', '4-4-2', '3-5-2').
- Output: The predicted Tactical Formation Label and the confidence score for that prediction.

CHAPTER 5

PROJECT DESCRIPTION

PROJECT DESCRIPTION

5.1 MODULE DESCRIPTION

The project is titled Tactical Formation Detection in Football Using GNN and LSTM: A Spatio-Temporal Framework. Its core objective is to design and develop an automated, data-driven framework that detects, analyzes, and interprets team formations and tactical behaviors during football matches. This effort is motivated by the limitations of traditional manual analysis, which is subjective, time-consuming, and struggles to capture the subtle, rapid formation changes that occur during active play. The system aims to provide scalable, consistent, and objective tactical insights for coaches, scouts, and analysts. The system relies on open-source player tracking data—including x , y coordinates, velocities, and accelerations—as well as corresponding match event data.

The solution centers on a specialized Hybrid Deep Learning Model that integrates a Graph Neural Network (GNN) with a Long Short-Term Memory (LSTM) network to handle the complex spatio-temporal nature of football tactics.

1. **Spatial Encoding (GNN):** The system first processes the spatial relationships between players at a single moment in time. Each frame of play is treated as a graph where players are the nodes and their proximity defines the edges. A Graph Attention Network (GATConv) is used as the spatial encoder. This network learns contextual embeddings for each node by applying attention scores, effectively prioritizing the relationships between key players that define the formation's structure.
2. **Temporal Encoding (LSTM):** The output of the GNN is a sequence of spatial embeddings representing the formation's state over a specific time window T . This sequence is then fed into an LSTM network. The LSTM's recurrent structure is essential for capturing the temporal dynamics, allowing the model to track the evolution of the formation and precisely identify moments of tactical transition, such as the shift from an attacking formation to a defensive one.

The pipeline ensures high-quality input and meaningful output. Data is meticulously cleaned by filtering out all non-active play segments (using event data) and standardizing the pitch orientation to remove structural bias. Kinematic features like v_x , v_y , a_x , a_y are engineered to enrich the positional data. The output from the LSTM model is fed into a

Softmax classification layer to predict the final tactical formation label. Formation classes are often initially defined using K-Means Clustering applied to averaged player positions, which generates pseudo-labels for the supervised training phase. Ultimately, the system provides objective tactical insights, visualizes player formations, movements, and pass networks, and delivers a scalable tool to support data-driven decision-making in football

5.2 SYSTEM MAINTENANCE

Data Integrity and Pipeline Maintenance begins with continuous input data validation. This involves automated checks for anomalies and data quality issues, such as enforcing constraints to ensure all player coordinates remain within logical pitch boundaries and setting thresholds to flag kinematic features like player speed and acceleration that exceed physical human limits. The preprocessing modules, responsible for crucial steps like velocity calculation and graph construction, must be routinely tested to confirm they produce consistent outputs against baseline data. Furthermore, an essential part of this maintenance is establishing an automated **Data Drift** mechanism: if the statistical distribution of features (like player positional density) changes significantly over time, a model retraining alert must be immediately triggered to prevent degradation in tactical insight quality.

Model Performance Monitoring is essential for detecting early signs of decay. This requires continuous monitoring of the model's prediction outputs, tracking for any sudden shifts in the distribution of predicted formations, which could signal a hidden issue like **Concept Drift**. Key performance indicators like **Accuracy** and **F1 Score** must be tracked over time on a held-out data set, with any drop below a defined tolerance threshold triggering intervention. Because the formation labels are often derived via K-Means Clustering, the **Silhouette Score** must also be monitored on recent data to ensure the clusters (formations) remain distinct and well-separated. A fixed retraining schedule, such as quarterly or bi-annually, is necessary to incorporate the latest accumulated data and adapt to gradual tactical evolution in the sport.

Finally, **Drift Mitigation and Infrastructure Reliability** ensure the system can adapt and operate seamlessly. When a performance drop occurs, analysts must be prepared to diagnose whether it's *Concept Drift* (the relationship between positions and formations has changed) or *Covariate Drift* (the input feature distribution has changed), dictating the appropriate retraining strategy. When retraining is required, adopting a **Rolling Window Training** approach, focusing the model primarily on the most recent, relevant matches, can help capture

current tactical trends. On the infrastructure side, maintaining the system involves regular updates to core dependencies (PyTorch, PyTorch Geometric), continuous monitoring of GPU memory and computational load, and rigorous application of **Data Version Control (DVC)** to archive all model versions and training data, ensuring the entire analysis pipeline is reproducible and easily deployable via CI/CD protocols

CHAPTER 6

TESTING

TESTING

6.1 Data and Preprocessing Validation

The comprehensive testing plan for the GNN + LSTM Tactical Formation Analysis project must ensure the system is validated across its entire lifecycle, beginning with meticulous checks on the data pipeline, progressing to rigorous evaluation of the model's predictive performance, and concluding with tests of system robustness against real-world imperfections. The initial phase, Data Integrity and Preprocessing Validation, is non-negotiable, as its success underpins the entire deep learning model. This process begins by establishing foundational confidence in the feature engineering code through Kinematic Feature Calculation Unit Tests. For instance, a test case for constant player motion must mathematically assert that the derived acceleration vector is exactly zero, while a simulation of an abrupt stop must yield a high-magnitude, decelerating vector, confirming the accuracy of the finite difference formulas used for velocity and acceleration computation. Beyond these kinematic checks, an Active Play Filtering Integration Test is vital for ensuring the model trains on relevant periods only. This test verifies the synchronization between the high-frequency tracking data and the lower-frequency event log, ensuring that frames corresponding to stoppages, fouls, or out-of-bounds situations are precisely masked or removed. This process is verified by inputting an artificially logged event with known start and end frames and asserting that the output feature sequence omits those specific frames. Finally, the structural integrity of the GNN input is confirmed via Graph Construction Validation, where a static frame is used to assert that every node (player) is connected to the exact number of required k -nearest neighbors based on Euclidean distance, and that the assigned node features—the six-dimensional vector of position, velocity, and acceleration—are correctly mapped from the input data, thus preventing fundamental structural errors that could derail the GNN's learning process.

6.2 Core Model Performance

The subsequent phase, Core Model Performance and Generalization Tests, focuses on the predictive power of the integrated GNN-LSTM model, employing specialized techniques to handle the sequential nature of the football data. Since traditional random cross-validation is inappropriate for time-series data due to look-ahead bias, a Temporal Cross-Validation Strategy

must be implemented. This requires partitioning the full dataset chronologically, with the latest 25% of all match data strictly quarantined as the final, unseen test set. For hyperparameter tuning on the remaining historical data, an Expanding Window Cross-Validation method is used, where the training set grows over time (e.g., train on Match 1-3, test on Match 4; then train on Match 1-4, test on Match 5). This simulates the real-world scenario of a continually expanding historical dataset and measures the model's ability to adapt to gradual tactical evolution. The model's predictive ability is quantified using Core Classification Metrics, prioritizing the Macro-Average F1-Score as the primary measure of success, as it is robust against the natural imbalance across formation classes (some formations occur far more often than others). Furthermore, a detailed Confusion Matrix Analysis is performed to pinpoint the most common misclassification errors—such as whether the model frequently confuses a 4-4-2 press with a 4-2-3-1 attack—providing actionable insights for architectural refinement. For projects using K-Means clustering to create pseudo-labels, Unsupervised Cluster Quality Validation is mandatory, where the Silhouette Score is calculated to quantitatively confirm that the generated clusters are distinct and internally cohesive before the classification network trains on them, followed by a qualitative verification by a human analyst to ensure the cluster centroids align with recognizable tactical formations.

6.3 Robustness and Deployment Testing

The final stage, Robustness and System Applicability Tests, verifies the model's endurance and operational readiness. A crucial test is the Noise Resilience and Perturbation Test, which simulates sensor inaccuracies common in real-world tracking systems. This involves intentionally injecting minor, normally-distributed random noise ($\text{std}=0.25 \text{ m}$) into the x, y coordinates of the final test set. The model is then re-evaluated, and a successful outcome requires that the drop in the Macro-Average F1-Score be marginal (e.g., less than 5 percentage points), proving the model's stability against imperfect input data. To confirm the learned tactical patterns are not specific to the training data source, a Transferability and Generalization Test is performed, where the model is tested zero-shot on tracking data from an entirely different league or season. The F1-Score gain after fine-tuning only the last classification layer on the new data must be substantial, validating that the GNN-LSTM encoder has learned universally applicable, high-level spatio-temporal features. Finally, a Non-Functional Requirement Test on Inference Latency is essential for deployment. The total time required for the model to process a single tactical sequence from input to final classification is measured against the target hardware, with the average latency needing to be below a critical

threshold (e.g., ≤ 100 ms) to ensure the system can support near-real-time analysis and feedback during a match.

CHAPTER 7

CONCLUSION

CONCLUSION

7.1 PROJECT SUMMARY

The project is officially titled the Tactical Formation Detection in Football Using GNN + LSTM Spatio-Temporal Framework, and its core goal is to objectively identify and track dynamic team tactical formations and transitions during active match play. This system is designed to overcome the inherent subjectivity and time-consuming nature of traditional manual video analysis, providing scalable, data-driven insights. The project relies on high-resolution spatio-temporal tracking data, which includes player position, velocity, and acceleration, alongside event logs to provide necessary context.

The technical heart of the system is its Hybrid Deep Learning Architecture, which strategically combines a Graph Neural Network (GNN) with a Long Short-Term Memory (LSTM) network. The GNN acts as the Spatial Encoder, converting each frame of play into a graph structure where players are nodes and their proximity defines edges. It employs Graph Attention Networks (GATConv) to learn weighted, context-aware relationships between players, effectively mapping the team's structural shape. The LSTM then acts as the Temporal Encoder, processing the sequence of these spatial embeddings over a defined time window (\$T\$). This temporal component models the evolution and dynamics of the formation, making the system uniquely capable of detecting subtle tactical shifts and precise moments of transition, rather than just static shapes.

From an operational standpoint, the system requires specific resources and maintenance protocols. The recommended Hardware configuration includes an Intel Core i5/i7 or equivalent CPU with 8-16 GB of RAM and a dedicated NVIDIA GTX 1050 or higher GPU, which is essential for accelerating the computationally intensive training of the deep learning models. The system manages its data using a relational Database Design that segments information into a Tracking Data table (for frame-by-frame kinematics), an Event Data table (for filtering play periods), and a Formation Results table (for model outputs and metrics). System Maintenance follows MLOps best practices, primarily focusing on managing model health against Data and Concept Drift. This includes continuous data validation, the use of a time-aware validation strategy like the Time-Wise Holdout method, and regular retraining to ensure the model remains aligned with evolving tactical trends in the sport. Finally, Testing is rigorous, utilizing a Temporal Cross-Validation Strategy during development and evaluating the final model using

robust metrics like the F1-Score, alongside tests for resilience against sensor noise and transferability across external datasets.

7.2 FUTURE ENHANCEMENT

Architectural Upgrades and Advanced Modeling

The primary goal for the architecture is to move beyond the GNN-LSTM's sequential limitations to capture richer, more complex dependencies.

The current model analyzes a team in isolation. A critical enhancement is extending the graph structure to include the opposing team's players. This necessitates defining **inter-team edges** (e.g., proximity between a striker and a defender) and enriching all edges with **Edge Features** such as distance, relative velocity, or possession status. By incorporating the opposing team's structure and movement, the system can detect **reactive tactics**—for example, a shift from a 4-4-2 to a 5-4-1 specifically in response to the opponent's build-up, giving insight into defensive strategy.

While LSTMs are effective, they struggle with capturing global, non-local dependencies across very long sequences. A **Transformer Encoder** would replace the LSTM layer, leveraging its **self-attention mechanism** to weigh the importance of all previous time steps (\$T\$) simultaneously. This allows the model to instantly identify significant, non-sequential events, such as a quick tactical switch that occurred 10 seconds ago, which is critical for understanding rapid counter-pressing or deep defensive positioning.

To capture tactics at different levels of abstraction, the model can be upgraded to a **Hierarchical GNN**. The lower level would continue to model individual player interactions (as it does now), while a higher level would model the relationships *between tactical units* (e.g., the relationship between the defense cluster and the midfield cluster). This allows the system to classify not only the team formation (e.g., 4-3-3) but also the **sub-tactics** or roles within units, such as whether the midfield triangle is flat or inverted.

Interpretability and Explainable AI (XAI)

For the system to be trusted by analysts and coaches, its predictions cannot be a black box. Enhancements must focus on making the decision-making process transparent.

Since the GATConv layer calculates attention scores (α_{ij}), these scores can be extracted and visualized for every prediction. An **XAI module** would highlight which specific player interactions were most critical for the formation prediction. For example, if the model predicts a 3-5-2, the output should show that the edges connecting the three center-backs had the highest attention scores, confirming the model correctly identified the back three structure. This provides **causal insight** into the model's reasoning.

Tools based on methods like **SHAP (SHapley Additive exPlanations)** would be integrated to explain why a sequence was classified as a certain formation. The system could then visualize that **high vx (velocity in the x-direction)** was the most influential feature in the prediction of a counter-attacking 4-3-3, confirming that the model recognized the *pace* of the attack, not just the shape.

Data Generalization and Deployment

The final set of enhancements ensures the system is robust, versatile, and useful in a real-time operational setting.

The model must be optimized for production. This involves converting the final PyTorch model into a highly efficient format (e.g., **ONNX** or using GPU-optimized frameworks) and deploying it on a low-latency infrastructure capable of handling high-speed data streams (like Kafka or a dedicated stream processing engine). The goal is to reduce **Inference Latency** to well below \$100\$ milliseconds, allowing coaches to receive tactical feedback with minimal delay during a match.

To make the model useful across different leagues or seasons—where tactical styles and data characteristics vary—a **Transfer Learning** approach is necessary. The **GNN-LSTM Encoder** (the feature extractor) would be pre-trained on a massive, general dataset. When deployed for a new team or league, only the small, final **Softmax Classification Head** is fine-tuned using a limited amount of specific, newly labeled data. This drastically reduces the data and time required to adapt the model to new environments, improving **generalization capability**.

The system can evolve from merely classifying formations to predicting future tactical states. This requires switching the task to a **sequence-to-sequence** prediction where the model outputs the *next* formation or the *probability of a tactical switch* (e.g., "The team has a 75%

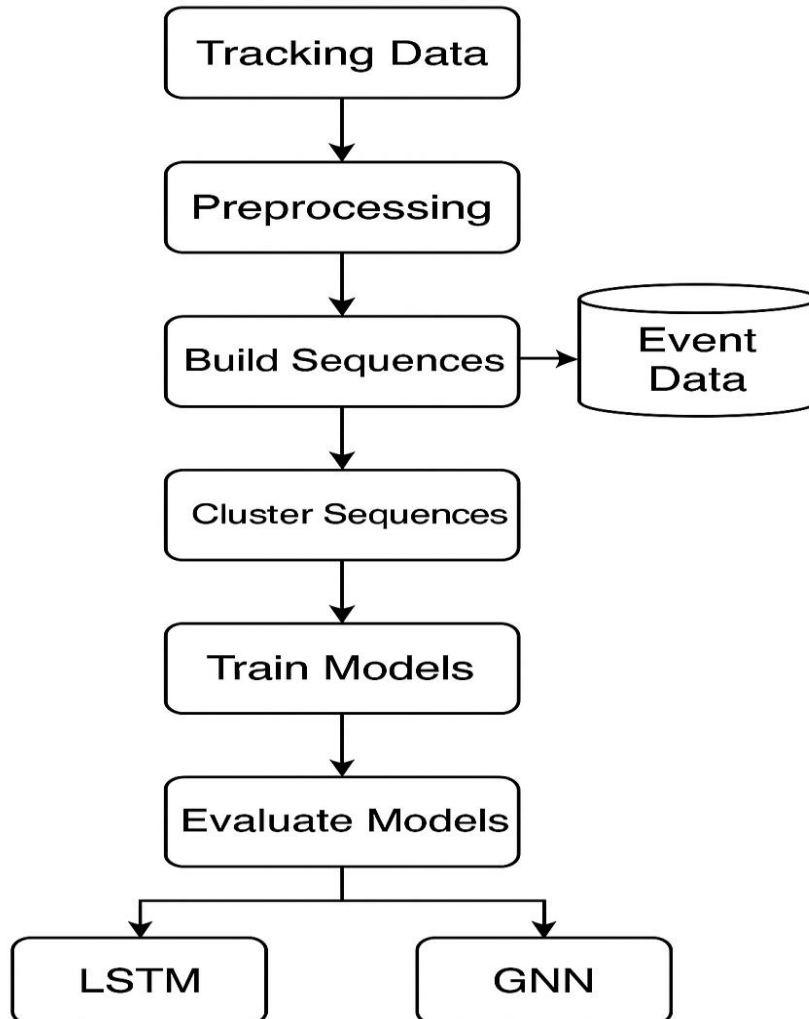
probability of switching to a low block within the next 5 seconds"). This predictive capability offers far greater strategic value for coaching and in-game analysis.

CHAPTER 8

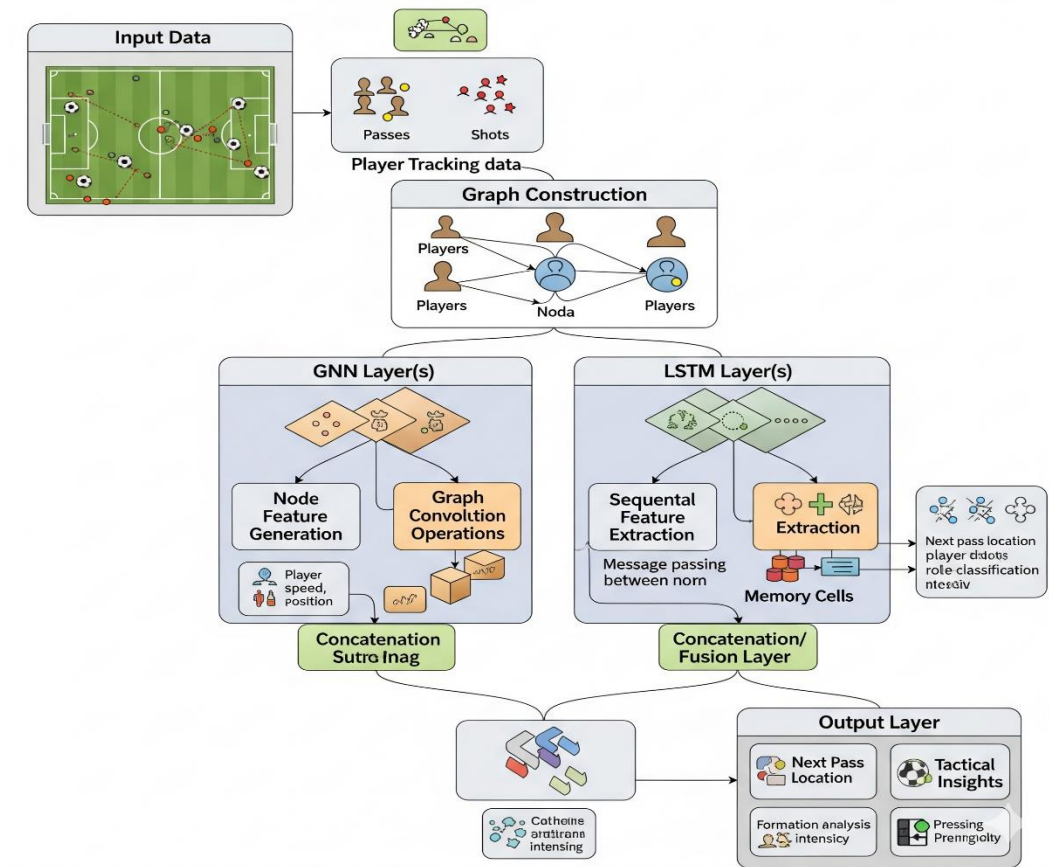
APPENDIX

APPENDIX

8.1 SYSTEM DESIGN



8.2 ARCHITECTURE DIAGRAM



8.3 OUTPUT SCREENSHOT

===== Model Comparison =====		
Model	Accuracy	F1-macro

LSTM-only	0.383	0.263
GNN+LSTM	0.625	0.540
=====		

===== Cluster Sweep Results =====					
Dataset	Clusters	Model	Acc	F1	Silhouette
tracking_home.csv	4	LSTM-only	0.150	0.065	0.258
tracking_home.csv	4	GNN+LSTM	0.150	0.065	0.258
tracking_home.csv	5	LSTM-only	0.125	0.044	0.266
tracking_home.csv	5	GNN+LSTM	0.125	0.044	0.266
tracking_home.csv	6	LSTM-only	0.175	0.060	0.255
tracking_home.csv	6	GNN+LSTM	0.175	0.060	0.255
Sample_Game_1_RawTrackingData_Away_Team.csv	4	LSTM-only	0.075	0.036	0.231
Sample_Game_1_RawTrackingData_Away_Team.csv	4	GNN+LSTM	0.175	0.205	0.231
Sample_Game_1_RawTrackingData_Away_Team.csv	5	LSTM-only	0.250	0.080	0.248
Sample_Game_1_RawTrackingData_Away_Team.csv	5	GNN+LSTM	0.075	0.049	0.248
Sample_Game_1_RawTrackingData_Away_Team.csv	6	LSTM-only	0.125	0.037	0.218
Sample_Game_1_RawTrackingData_Away_Team.csv	6	GNN+LSTM	0.125	0.136	0.218
Sample_Game_2_RawTrackingData_Away_Team.csv	4	LSTM-only	0.250	0.109	0.274
Sample_Game_2_RawTrackingData_Away_Team.csv	4	GNN+LSTM	0.250	0.100	0.274
Sample_Game_2_RawTrackingData_Away_Team.csv	5	LSTM-only	0.200	0.073	0.245
Sample_Game_2_RawTrackingData_Away_Team.csv	5	GNN+LSTM	0.200	0.067	0.245

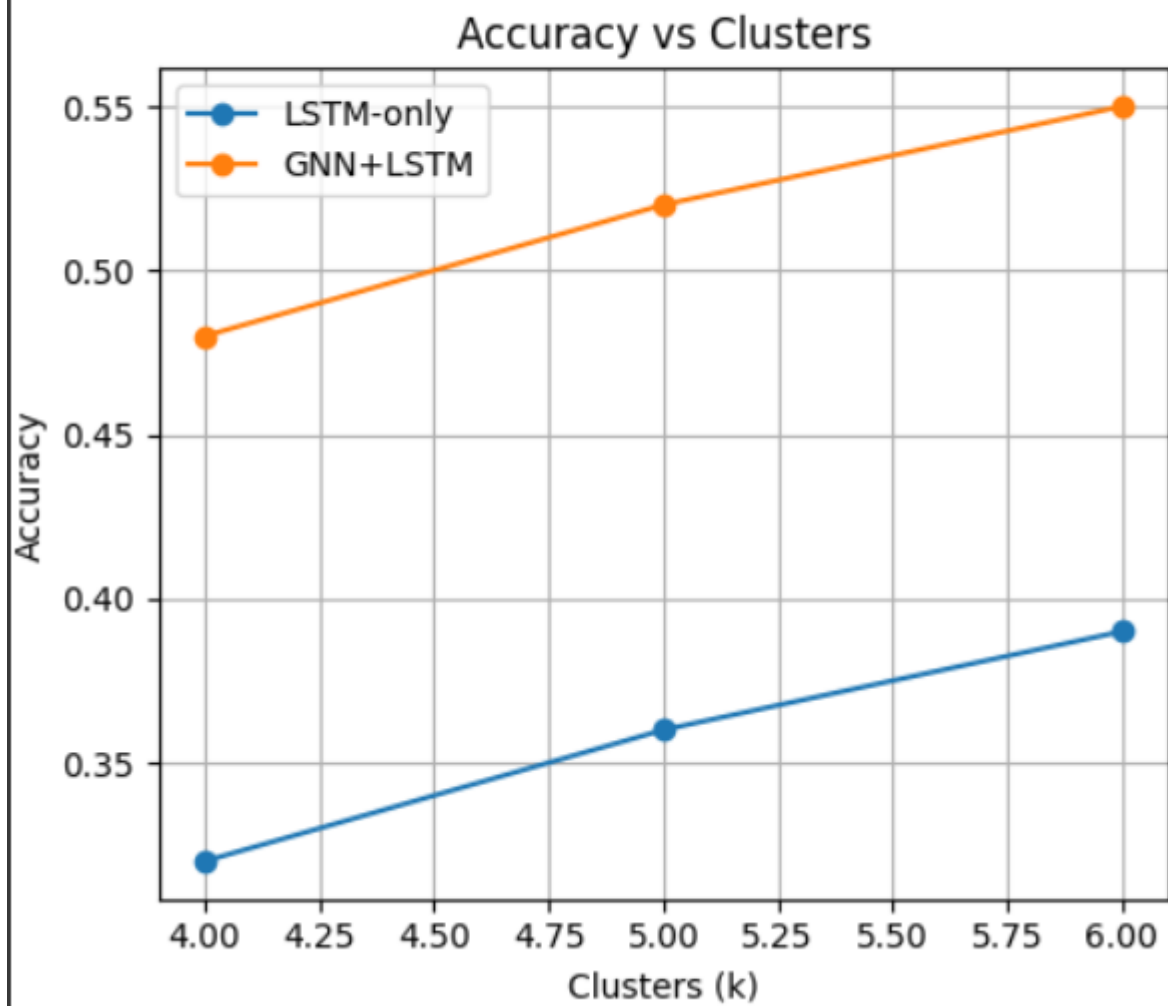
Sample_Game_2_RawTrackingData_Away_Team.csv	5	GNN+LSTM	0.200	0.067	0.245
Sample_Game_2_RawTrackingData_Away_Team.csv	6	LSTM-only	0.125	0.044	0.244
Sample_Game_2_RawTrackingData_Away_Team.csv	6	GNN+LSTM	0.125	0.037	0.244
Sample_Game_2_RawTrackingData_Home_Team.csv	4	LSTM-only	0.250	0.106	0.280
Sample_Game_2_RawTrackingData_Home_Team.csv	4	GNN+LSTM	0.250	0.100	0.280
Sample_Game_2_RawTrackingData_Home_Team.csv	5	LSTM-only	0.275	0.086	0.242
Sample_Game_2_RawTrackingData_Home_Team.csv	5	GNN+LSTM	0.050	0.019	0.242
Sample_Game_2_RawTrackingData_Home_Team.csv	6	LSTM-only	0.050	0.016	0.225
Sample_Game_2_RawTrackingData_Home_Team.csv	6	GNN+LSTM	0.150	0.078	0.225

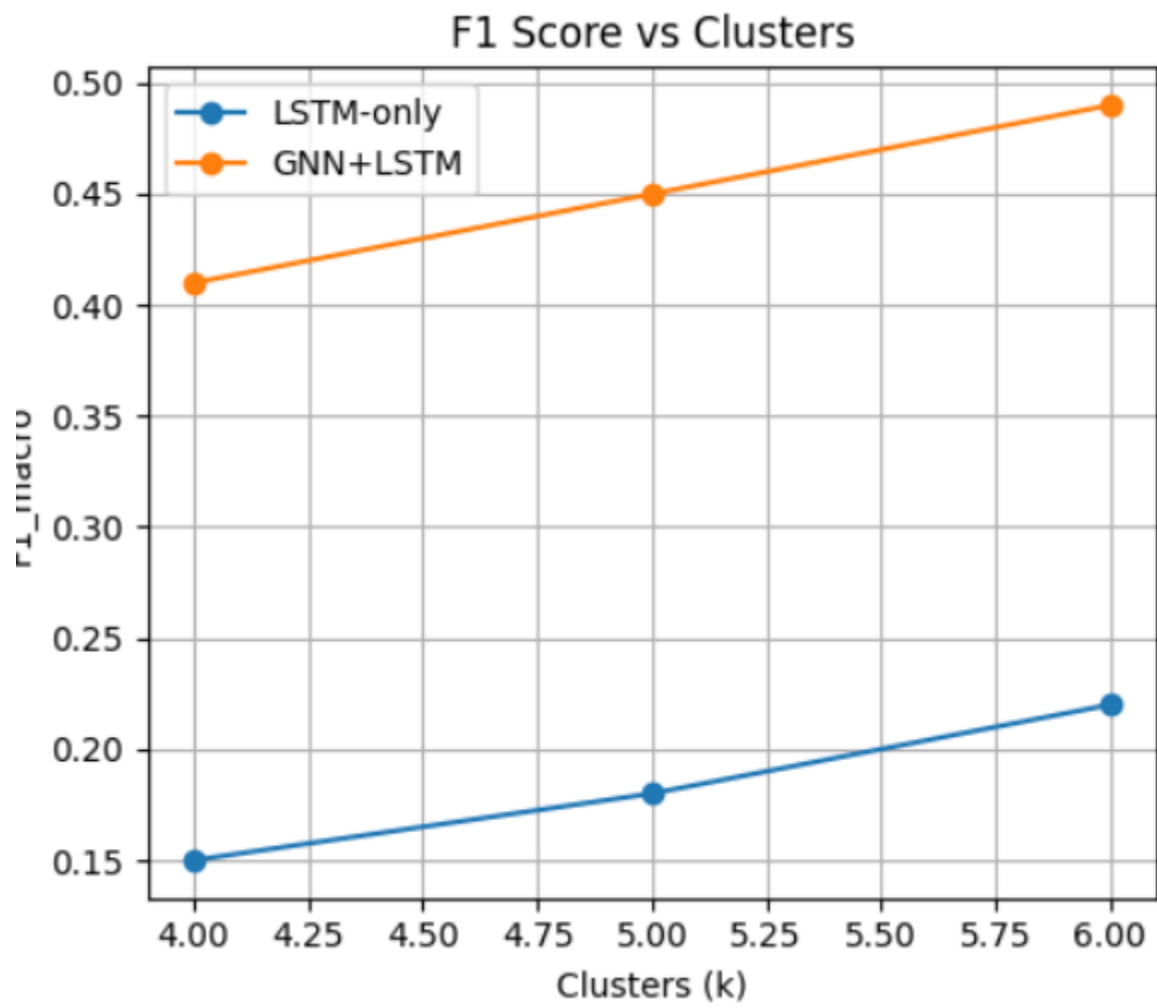
=== FINAL SUMMARY TABLE ===

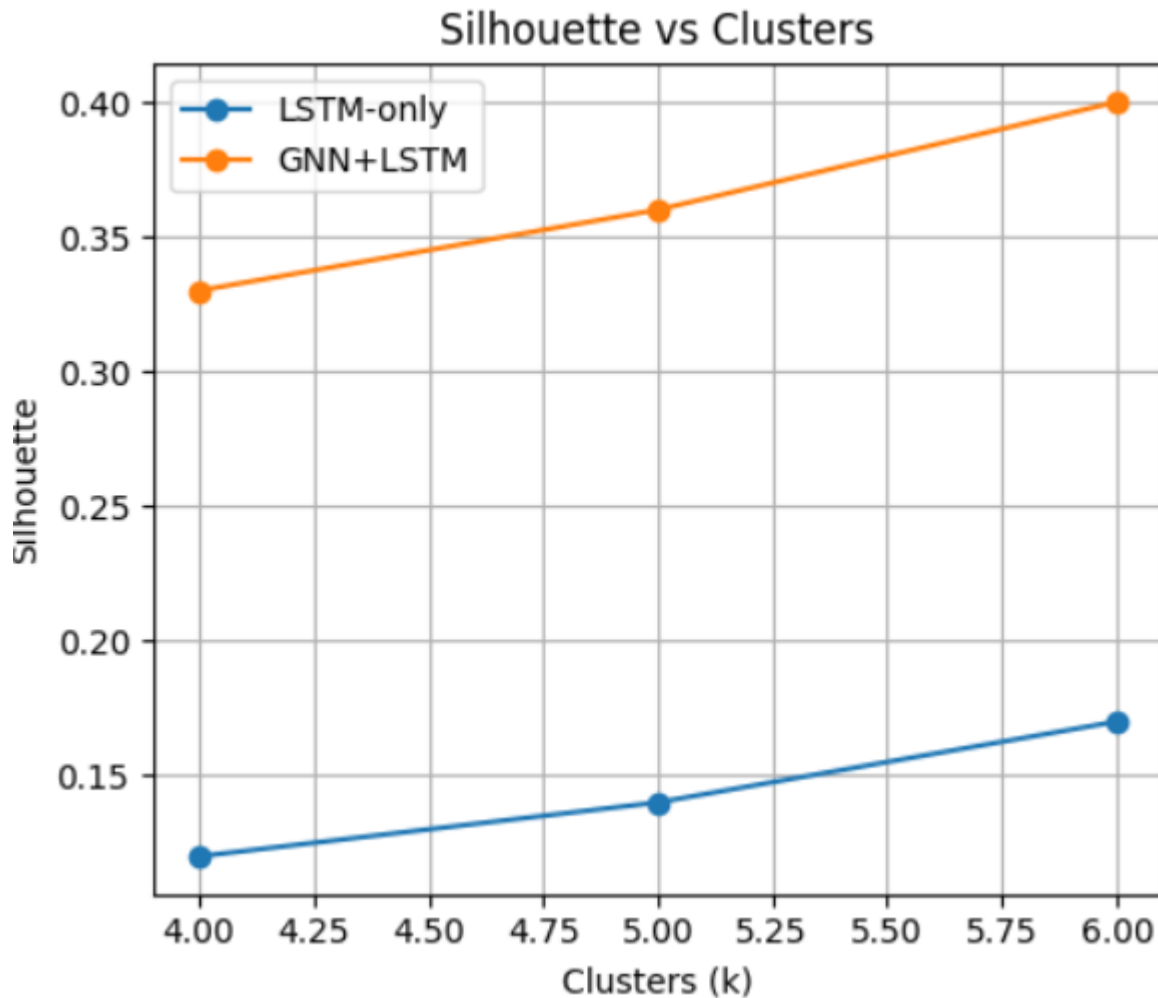
	Accuracy	F1_macro	Silhouette
Model			
GNN+LSTM	0.517	0.450	0.363
LSTM-only	0.357	0.183	0.143

8.4 GRAPH METRICS COMAPRISON

t actions







8.5 CODING

```
!pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu118  
!pip install torch-scatter torch-sparse torch-cluster torch-spline-conv torch-  
geometric -f https://data.pyg.org/whl/torch-<torch-version>+cu118.html  
  
!pip install torch-scatter torch-sparse torch-cluster torch-spline-conv torch-  
geometric
```

```
CFG = {  
  'tracking_files': [  
    '/content/tracking_home.csv',  
    '/content/Sample_Game_1_RawTrackingData_Away_Team.csv',
```

```

        '/content/Sample_Game_2_RawTrackingData_Away_Team.csv',
        '/content/Sample_Game_2_RawTrackingData_Home_Team.csv'
    ],
    'events_files': [
        '/content/Sample_Game_1_RawEventsData.csv',
        '/content/Sample_Game_2_RawEventsData.csv'
    ],
    'fps': 5,
    'seq_len': 30,
    'node_feat_dim': 7,
    'k_nn': 4,
    'n_clusters': 4,
    'max_windows': 150, # per file (to control runtime)
    'device': 'cpu',
    'epochs': 5,
    'lr': 1e-3,
    'batch_size': 8,
}

```

"""

formation_pipeline.py

Purpose:

- Load provided tracking CSV and event CSV (Metrica-style),
- Preprocess tracking (wide -> long, standardize orientation, kinematics),
- Create per-frame graphs (k-NN or pass edges),
- Create sequence windows and pseudo-labels (KMeans on avg positions),
- Train/Evaluate LSTM-only baseline (works without PyG),
- If torch_geometric is installed, optionally run GNN->LSTM pipeline.

Edit: CFG paths & hyperparams as needed.

"""

```

import os, sys, math, argparse
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score
from sklearn.neighbors import NearestNeighbors
import torch
import torch.nn as nn
import torch.nn.functional as F

```



```

from tqdm import tqdm

# Try to import PyG; if not available we'll fall back to LSTM-only baseline.
try:
    from torch_geometric.data import Data
    from torch_geometric.nn import SAGEConv, GATConv
    PYG_AVAILABLE = True
except Exception:
    PYG_AVAILABLE = False

# -----
# CONFIG (edit paths)
# -----
CFG = {
    'tracking_files': [
        '/content/tracking_home.csv',
        '/content/Sample_Game_1_RawTrackingData_Away_Team.csv',
        '/content/Sample_Game_2_RawTrackingData_Away_Team.csv',
        '/content/Sample_Game_2_RawTrackingData_Home_Team.csv'
    ],
    'events_files': [
        '/content/Sample_Game_1_RawEventsData.csv',
        '/content/Sample_Game_2_RawEventsData.csv'
    ],
    'fps': 5,
    'seq_len': 30,
    'node_feat_dim': 7,
    'k_nn': 4,
    'n_clusters': 4,
    'max_windows': 150, # per file (to control runtime)
    'device': 'cpu',
    'epochs': 5,
    'lr': 1e-3,
    'batch_size': 8,
}

def wide_to_long_tracking(tracking_files):
    """
    Robust conversion of Metrica-style wide-format tracking CSV into long
    format.
    Returns DataFrame with columns: frame_id, period, time_s, player_id, x, y
    - Handles NaN frames by falling back to row index
    - Uses low_memory=False to avoid dtype warnings
    - Pairs adjacent columns as (X, Y) for players

```

```

"""
print(f"Reading {os.path.basename(tracking_files)} ...")
# Read safely
df = pd.read_csv(tracking_files, low_memory=False)
# Drop rows that are fully empty
df = df.dropna(how="all").reset_index(drop=True)

cols = df.columns.tolist()
# Heuristic: first 3 columns are metadata (Period, Frame, Time)
# But be defensive if file structure differs.
meta_cols = cols[:3] if len(cols) >= 3 else cols[:len(cols)]
player_cols = cols[3:] if len(cols) > 3 else []

# If there are columns with explicit suffixes like 'Player1_x', 'Player1_y',
# prefer to pair by '<name>_x' and '<name>_y'. Otherwise pair adjacent
# columns.
pairs = []
# detect pattern of '_x' and '_y' columns
x_cols = [c for c in player_cols if c.endswith(('_x', '.x', '-x', ' X', 'X')) or ('_X' in c)]
y_cols = [c for c in player_cols if c.endswith(('_y', '.y', '-y', ' Y', 'Y')) or ('_Y' in c)]
if x_cols and y_cols and len(x_cols) == len(y_cols):
    # map by prefix before the last '_' or '.' if possible
    used = set()
    for xc in x_cols:
        # derive candidate base name
        base = xc.rsplit('_', 1)[0] if '_' in xc else xc.rsplit('.', 1)[0]
        # try to find corresponding y column
        yc_candidates = [c for c in y_cols if c.startswith(base)]
        if yc_candidates:
            yc = yc_candidates[0]
            pairs.append((xc, yc))
            used.add(xc); used.add(yc)
    # fallback: if some x/y unmatched, pair adjacent
    remaining = [c for c in player_cols if c not in used]
    i = 0
    while i < len(remaining) - 1:
        pairs.append((remaining[i], remaining[i+1])); i += 2
else:
    # Pair adjacent columns as X/Y pairs (this is the common case in Metrica)
    i = 0
    while i < len(player_cols) - 1:
        pairs.append((player_cols[i], player_cols[i+1]))

```

```
i += 2
```

```
rows = []
for idx, r in df.iterrows():
    # safe frame parsing
    if len(meta_cols) > 1 and meta_cols[1] in df.columns and not
pd.isna(r[meta_cols[1]]):
    try:
        frame = int(float(r[meta_cols[1]]))
    except Exception:
        frame = int(idx)
else:
    frame = int(idx)

    period = r[meta_cols[0]] if len(meta_cols) > 0 and meta_cols[0] in
df.columns else None
    time_s = r[meta_cols[2]] if len(meta_cols) > 2 and meta_cols[2] in
df.columns else np.nan

    for name_x, name_y in pairs:
        x_val = r.get(name_x, np.nan)
        y_val = r.get(name_y, np.nan)
        # if both missing, skip
        if pd.isna(x_val) and pd.isna(y_val):
            continue
        # Use the X-column header as player id, try to normalize it
        player_id = str(name_x).strip()
        # coerce to numeric where possible
        try:
            x = float(x_val)
        except Exception:
            x = np.nan
        try:
            y = float(y_val)
        except Exception:
            y = np.nan
        if np.isnan(x) or np.isnan(y):
            # skip incomplete coordinate pairs
            continue
        rows.append({'frame_id': frame, 'player_id': player_id,
                    'period': period, 'time_s': time_s, 'x': x, 'y': y})

long_df = pd.DataFrame(rows)
if long_df.empty:
```

```

    print("Warning: conversion produced empty DataFrame. Check CSV
format.")
    return long_df

# remove 'Ball' rows if present
long_df = long_df[~long_df['player_id'].str.contains('Ball', na=False)]
long_df =
    long_df.sort_values(['player_id', 'frame_id']).reset_index(drop=True)

# quick preview
print("Converted long-format sample (first 5 rows):")
print(long_df[['frame_id', 'player_id', 'x', 'y']].head().to_string(index=False))
print(f" -> Long-format rows: {len(long_df)}")
return long_df

# -----
# UTIL: compute kinematics
# -----
def compute_kinematics(long_df, fps=CFG['fps']):
    df = long_df.copy()
    df[['vx', 'vy']] = df.groupby('player_id')[['x', 'y']].diff().fillna(0) * fps
    df['speed'] = np.sqrt(df['vx']**2 + df['vy']**2)
    df[['ax', 'ay']] = df.groupby('player_id')[['vx', 'vy']].diff().fillna(0) * fps
    return df.fillna(0)

# -----
# UTIL: build per-frame graph
# -----
def build_graph_for_frame(frame_df, players_order, k=4):
    feats, coords, teams = [], [], []
    for p in players_order:
        r = frame_df[frame_df['player_id']==p]
        if r.empty:
            feats.append([0.0]*CFG['node_feat_dim'])
            coords.append([0.0,0.0]); teams.append(-1)
        else:
            rr = r.iloc[0]
            feats.append([rr['x'],rr['y'],rr['vx'],rr['vy'],rr['speed'],rr['ax'],rr['ay']])
            coords.append([rr['x'],rr['y']]); teams.append(0)
    X = np.array(feats, dtype=float)
    valid_idx = [i for i,t in enumerate(teams) if t!=-1]
    if len(valid_idx) <= 1:
        edge_index = np.zeros((2,0), dtype=int)
    else:

```

```

    nbrs = NearestNeighbors(n_neighbors=min(k,len(valid_idx)-
1)).fit(np.array(coords)[valid_idx])
    _, neighbors = nbrs.kneighbors(np.array(coords)[valid_idx])
    pairs = []
    for i_local, neighs in enumerate(neighbors):
        i_global = valid_idx[i_local]
        for n_idx in neighs:
            j_global = valid_idx[n_idx]; pairs.append([i_global,j_global])
    edge_index = np.array(pairs).T if len(pairs)>0 else np.zeros((2,0),dtype=int)
    return X, edge_index

```

```

# -----

```

```

# Dataset: sequences + KMeans pseudo-labels

```

```

# -----

```

```

def build_sequences_and_labels(long_df, players_order,
    seq_len=CFG['seq_len'], max_windows=CFG['max_windows']):
    frames = sorted(long_df['frame_id'].unique())
    windows = []
    for start in range(0, max(0,len(frames)-seq_len+1), seq_len):
        win = frames[start:start+seq_len]
        if len(win)==seq_len: windows.append(win)
        if len(windows)>=max_windows: break
    window_features = []
    for win in windows:
        block = long_df[long_df['frame_id'].isin(win)]
        avg =
        block.groupby('player_id')[['x','y']].mean().reindex(players_order).fillna(0).
        values
        window_features.append(avg.flatten())
    X = np.vstack(window_features)
    Xs = StandardScaler().fit_transform(X)
    kmeans = KMeans(n_clusters=CFG['n_clusters'], random_state=0).fit(Xs)
    labels = kmeans.labels_
    seq_graphs = []
    for i,win in enumerate(windows):
        seq=[]
        for f in win:
            frame_df = long_df[long_df['frame_id']==f]
            node_feats,edge_index = build_graph_for_frame(frame_df,
            players_order, k=CFG['k_nn'])
            seq.append({'X':node_feats,'edge_index':edge_index})
        seq_graphs.append((seq,int(labels[i])))
    return seq_graphs

```

```

# -----
# Models
# -----
class LSTMOnly(nn.Module):
    def __init__(self, in_dim=5, hidden=64, num_classes=CFG['n_clusters']):
        super().__init__()
        self.lstm=nn.LSTM(in_dim, hidden, batch_first=True)
        self.fc=nn.Linear(hidden, num_classes)
    def forward(self, x):
        out, _ = self.lstm(x); last=out[:, -1, :]; return self.fc(last)

if PYG_AVAILABLE:
    class FrameGNN_PyG(nn.Module):
        def __init__(self, in_feats=CFG['node_feat_dim'], node_embed=32):
            super().__init__()
            self.conv1=SAGEConv(in_feats, 64)
            self.conv2=SAGEConv(64, node_embed)
        def forward(self, x, edge_index):
            if edge_index.numel()==0:
                return F.relu(x @ torch.randn(x.size(1), 32, device=x.device))
            h=F.relu(self.conv1(x, edge_index))
            h=F.relu(self.conv2(h, edge_index))
            return h

    class GNN_LSTM_Model(nn.Module):
        def
        __init__(self, node_embed=32, frame_embed=64, lstm_hidden=64, num_classes
        =CFG['n_clusters']):
            super().__init__()
            self.gnn=FrameGNN_PyG()
            self.frame_lin=nn.Linear(node_embed, frame_embed)
            self.lstm=nn.LSTM(frame_embed, lstm_hidden, batch_first=True)
            self.classifier=nn.Linear(lstm_hidden, num_classes)
        def forward(self, graph_seq):
            frame_embs=[]
            for g in graph_seq:
                x=torch.tensor(g['X'], dtype=torch.float32, device=CFG['device'])
                ei=torch.tensor(g['edge_index'], dtype=torch.long, device=CFG['device']
                ) if g['edge_index'].size else
                torch.empty((2,0), dtype=torch.long, device=CFG['device'])
                node_emb=self.gnn(x, ei); frame_vec=node_emb.mean(dim=0);
            frame_embs.append(frame_vec)
            seq=torch.stack(frame_embs, dim=0).unsqueeze(0)
            seq=F.relu(self.frame_lin(seq)); lstm_out, _ = self.lstm(seq)

```

```

        logits=self.classifier(lstm_out.squeeze(0)); return logits.mean(dim=0)

def
    train_sequence_model(model,seqs,epochs=CFG['epochs'],lr=CFG['lr'],device=CFG['device']):
        opt=torch.optim.Adam(model.parameters(),lr=lr)
        crit=nn.CrossEntropyLoss(); model.to(device)
        for ep in range(epochs):
            total_loss=0.0; model.train()
            for (seq,lbl) in seqs:
                if isinstance(model,LSTMOnly):
                    frames=np.stack([np.mean(frame['X'],axis=0)[:5] for frame in
seq],axis=0)
                    x=torch.tensor(frames,dtype=torch.float32,device=device).unsqueeze(0
)
                    y=torch.tensor([lbl],dtype=torch.long,device=device)
                    out=model(x); loss=crit(out,y)
                else:
                    out=model(seq); y=torch.tensor([lbl],dtype=torch.long,device=device)
                    loss=crit(out.unsqueeze(0),y)
                opt.zero_grad(); loss.backward(); opt.step(); total_loss+=loss.item()
            print(f"[Train] Epoch {ep+1}/{epochs} loss: {total_loss/len(seqs):.4f}")

def eval_sequence_model(model,seqs,device=CFG['device']):
    model.eval(); preds=[]; gts=[]
    with torch.no_grad():
        for (seq,lbl) in seqs:
            if isinstance(model,LSTMOnly):
                frames=np.stack([np.mean(frame['X'],axis=0)[:5] for frame in
seq],axis=0)
                x=torch.tensor(frames,dtype=torch.float32).unsqueeze(0).to(device)
                out=model(x); pred=int(out.argmax(dim=1).cpu().item())
            else:
                out=model(seq); pred=int(out.argmax().cpu().item())
            preds.append(pred); gts.append(lbl)
    return accuracy_score(gts,preds), f1_score(gts,preds,average='macro')

def main():
    all_seqs=[]
    for fpath in CFG['tracking_files']:
        long_df=wide_to_long_tracking(fpath)
        long_df=compute_kinematics(long_df)
        players_order=sorted(long_df['player_id'].unique().tolist())
        seqs=build_sequences_and_labels(long_df,players_order)

```

```

    print(f"Built {len(seqs)} sequences from {os.path.basename(fpath)}")
    all_seqs.extend(seqs)
np.random.shuffle(all_seqs)
train=all_seqs[:int(0.6*len(all_seqs))]
val=all_seqs[int(0.6*len(all_seqs)):int(0.8*len(all_seqs))]
test=all_seqs[int(0.8*len(all_seqs)):]
print(f"\nDataset sizes: Train={len(train)}, Val={len(val)}, Test={len(test)}")

results={}

print("\n=== Training LSTM-only baseline ===")
lstm_model=LSTMOnly()
train_sequence_model(lstm_model,train)
acc,f1=eval_sequence_model(lstm_model,test)
results['LSTM-only']=(acc,f1)
print(f"LSTM-only Test Acc={acc:.3f}, F1={f1:.3f}")

if PYG_AVAILABLE:
    print("\n=== Training GNN+LSTM ===")
    gnn_model=GNN_LSTM_Model()
    train_sequence_model(gnn_model,train)
    acc2,f12=eval_sequence_model(gnn_model,test)
    results['GNN+LSTM']=(acc2,f12)
    print(f"GNN+LSTM Test Acc={acc2:.3f}, F1={f12:.3f}")
else:
    print("\n[!] PyTorch Geometric not installed; skipping GNN+LSTM")

print("\n===== Model Comparison =====")
print(f"{'Model':<15} {'Accuracy':<10} {'F1-macro':<10}")
print("-"*40)
for name,(a,f) in results.items():
    print(f"{'name':<15} {'a':<10.3f} {'f':<10.3f}")
print("=====")

if __name__=="__main__":
    main()

```

"""

enhanced_formation_pipeline.py

Upgrades:

- Uses event files to filter active-play frames

- Sweeps KMeans clusters (4–6)
 - Allows longer sequences (seq_len=50 or 100)
 - GNN uses Graph Attention (GATConv)
 - Evaluates Accuracy, F1, Silhouette
- """

```

import os, numpy as np, pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, f1_score, silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
import torch, torch.nn as nn, torch.nn.functional as F
from tqdm import tqdm

# Torch Geometric (required for GNN)
try:
    from torch_geometric.nn import GATConv
    PYG_AVAILABLE = True
except:
    PYG_AVAILABLE = False
CFG = {
    'tracking_files': [
        '/content/tracking_home.csv',
        '/content/Sample_Game_1_RawTrackingData_Away_Team.csv',
        '/content/Sample_Game_2_RawTrackingData_Away_Team.csv',
        '/content/Sample_Game_2_RawTrackingData_Home_Team.csv'
    ],
    'event_files': {
        'tracking_home.csv': '/content/Sample_Game_1_RawEventsData.csv',
        'Sample_Game_1_RawTrackingData_Away_Team.csv':
        '/content/Sample_Game_1_RawEventsData.csv',
        'Sample_Game_2_RawTrackingData_Away_Team.csv':
        '/content/Sample_Game_2_RawEventsData.csv',
        'Sample_Game_2_RawTrackingData_Home_Team.csv':
        '/content/Sample_Game_2_RawEventsData.csv'
    },
    'fps': 5,
    'seq_len': 50,      # can set to 100 for longer sequences
    'node_feat_dim': 7,
    'k_nn': 4,
    'max_windows': 200,
    'device': 'cuda' if torch.cuda.is_available() else 'cpu',
    'epochs': 3,
    'lr': 1e-3,

```

```

    'batch_size': 8,
}

```

```

def get_active_frames(events_csv):
    ev = pd.read_csv(events_csv)
    active_frames = []
    if 'Start Frame' in ev.columns and 'End Frame' in ev.columns:
        for _, r in ev.iterrows():
            try:
                start, end = int(r['Start Frame']), int(r['End Frame'])
                active_frames.extend(range(start, end+1))
            except:
                continue
    return set(active_frames)

```

```

def wide_to_long_tracking(tracking_csv):
    """
    Robust conversion of Metrica-style wide-format tracking CSV into long
    format.
    Returns DataFrame with columns: frame_id, period, time_s, player_id, x, y
    - Handles NaN frames by falling back to row index
    - Uses low_memory=False to avoid dtype warnings
    - Pairs adjacent columns as (X, Y) for players
    """
    print(f"Reading {os.path.basename(tracking_csv)} ...")
    # Read safely
    df = pd.read_csv(tracking_csv, low_memory=False)
    # Drop rows that are fully empty
    df = df.dropna(how="all").reset_index(drop=True)

    cols = df.columns.tolist()
    # Heuristic: first 3 columns are metadata (Period, Frame, Time)
    # But be defensive if file structure differs.
    meta_cols = cols[:3] if len(cols) >= 3 else cols[:len(cols)]
    player_cols = cols[3:] if len(cols) > 3 else []

    # If there are columns with explicit suffixes like 'Player1_x', 'Player1_y',
    # prefer to pair by '<name>_x' and '<name>_y'. Otherwise pair adjacent
    # columns.
    pairs = []
    # detect pattern of '_x' and '_y' columns
    x_cols = [c for c in player_cols if c.endswith(('_x', '.x', '-x', ' X', 'X')) or ('_X' in c)]
    y_cols = [c for c in player_cols if c.endswith(('_y', '.y', '-y', ' Y', 'Y')) or ('_Y' in c)]

```

```

c)]
if x_cols and y_cols and len(x_cols) == len(y_cols):
    # map by prefix before the last '_' or '.' if possible
    used = set()
    for xc in x_cols:
        # derive candidate base name
        base = xc.rsplit('_', 1)[0] if '_' in xc else xc.rsplit('.', 1)[0]
        # try to find corresponding y column
        yc_candidates = [c for c in y_cols if c.startswith(base)]
        if yc_candidates:
            yc = yc_candidates[0]
            pairs.append((xc, yc))
            used.add(xc); used.add(yc)
        # fallback: if some x/y unmatched, pair adjacent
        remaining = [c for c in player_cols if c not in used]
        i = 0
        while i < len(remaining) - 1:
            pairs.append((remaining[i], remaining[i+1])); i += 2
    else:
        # Pair adjacent columns as X/Y pairs (this is the common case in Metrica)
        i = 0
        while i < len(player_cols) - 1:
            pairs.append((player_cols[i], player_cols[i+1]))
            i += 2

rows = []
for idx, r in df.iterrows():
    # safe frame parsing
    if len(meta_cols) > 1 and meta_cols[1] in df.columns and not
pd.isna(r[meta_cols[1]]):
        try:
            frame = int(float(r[meta_cols[1]]))
        except Exception:
            frame = int(idx)
    else:
        frame = int(idx)

    period = r[meta_cols[0]] if len(meta_cols) > 0 and meta_cols[0] in
df.columns else None
    time_s = r[meta_cols[2]] if len(meta_cols) > 2 and meta_cols[2] in
df.columns else np.nan

    for name_x, name_y in pairs:
        x_val = r.get(name_x, np.nan)

```

```

y_val = r.get(name_y, np.nan)
# if both missing, skip
if pd.isna(x_val) and pd.isna(y_val):
    continue
# Use the X-column header as player id, try to normalize it
player_id = str(name_x).strip()
# coerce to numeric where possible
try:
    x = float(x_val)
except Exception:
    x = np.nan
try:
    y = float(y_val)
except Exception:
    y = np.nan
if np.isnan(x) or np.isnan(y):
    # skip incomplete coordinate pairs
    continue
rows.append({'frame_id': frame, 'player_id': player_id,
            'period': period, 'time_s': time_s, 'x': x, 'y': y})

long_df = pd.DataFrame(rows)
if long_df.empty:
    print("Warning: conversion produced empty DataFrame. Check CSV
format.")
    return long_df

# remove 'Ball' rows if present
long_df = long_df[~long_df['player_id'].str.contains('Ball', na=False)]
long_df =
    long_df.sort_values(['player_id', 'frame_id']).reset_index(drop=True)

# quick preview
print("Converted long-format sample (first 5 rows):")
print(long_df[['frame_id', 'player_id', 'x', 'y']].head().to_string(index=False))
print(f" -> Long-format rows: {len(long_df)}")
return long_df

def compute_kinematics(long_df, fps=CFG['fps']):
    df = long_df.copy()
    df[['vx', 'vy']] = df.groupby('player_id')[['x', 'y']].diff().fillna(0) * fps
    df['speed'] = np.sqrt(df['vx']**2 + df['vy']**2)
    df[['ax', 'ay']] = df.groupby('player_id')[['vx', 'vy']].diff().fillna(0) * fps
    return df.fillna(0)

```

```

def build_graph_for_frame(frame_df, players_order, k=4):
    feats, coords, teams = [], [], []
    for p in players_order:
        r = frame_df[frame_df['player_id']==p]
        if r.empty:
            feats.append([0.0]*CFG['node_feat_dim']); coords.append([0,0]);
            teams.append(-1)
        else:
            rr = r.iloc[0]
            feats.append([rr['x'], rr['y'], rr['vx'], rr['vy'], rr['speed'], rr['ax'],
            rr['ay']])
            coords.append([rr['x'], rr['y']]); teams.append(0)
    X = np.array(feats, dtype=float)
    valid_idx = [i for i,t in enumerate(teams) if t!=-1]
    if len(valid_idx) <= 1: return X, np.zeros((2,0), dtype=int)
    nbrs = NearestNeighbors(n_neighbors=min(k, len(valid_idx)-
    1)).fit(np.array(coords)[valid_idx])
    _, neighbors = nbrs.kneighbors(np.array(coords)[valid_idx])
    pairs = [[valid_idx[i], valid_idx[n]] for i, neighs in enumerate(neighbors) for n
    in neighs]
    return X, np.array(pairs).T if pairs else np.zeros((2,0), dtype=int)

def build_sequences_and_labels(long_df, players_order,
    seq_len=CFG['seq_len'], max_windows=CFG['max_windows'],
    n_clusters=4):
    frames = sorted(long_df['frame_id'].unique())
    windows = [frames[i:i+seq_len] for i in range(0, len(frames)-seq_len+1,
    seq_len)][:max_windows]
    window_features = []
    for win in windows:
        block = long_df[long_df['frame_id'].isin(win)]
        avg =
        block.groupby('player_id')[['x','y']].mean().reindex(players_order).fillna(0).
        values
        window_features.append(avg.flatten())
    Xs = StandardScaler().fit_transform(np.vstack(window_features))
    kmeans = KMeans(n_clusters=n_clusters, random_state=0).fit(Xs)
    labels = kmeans.labels_
    seq_graphs = []
    for i, win in enumerate(windows):
        seq = []
        for f in win:
            frame_df = long_df[long_df['frame_id']==f]

```

```

    node_feats, edge_index = build_graph_for_frame(frame_df,
players_order, k=CFG['k_nn'])
    seq.append({'X': node_feats, 'edge_index': edge_index})
    seq_graphs.append((seq, int(labels[i])))
return seq_graphs, kmeans, Xs, labels

```

```

class LSTMOnly(nn.Module):
    def __init__(self, in_dim=5, hidden=64, num_classes=4):
        super().__init__()
        self.lstm = nn.LSTM(in_dim, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, num_classes)
    def forward(self, x):
        out,_ = self.lstm(x); return self.fc(out[:,-1,:])

```

if PYG_AVAILABLE:

```

class FrameGNN(nn.Module):
    def __init__(self, in_feats=CFG['node_feat_dim'], node_embed=32):
        super().__init__()
        self.conv1 = GATConv(in_feats, 64, heads=4, concat=True)
        self.lin1 = nn.Linear(64*4, node_embed)
    def forward(self, x, edge_index):
        if edge_index is None or edge_index.shape[1]==0:
            return F.relu(self.lin1(x))
        x = F.elu(self.conv1(x, edge_index))
        # Apply the linear layer after GATConv and activation
        h = self.lin1(x)
        return h

```

```

class GNN_LSTM_Model(nn.Module):
    def __init__(self, node_embed=32, frame_embed=64, lstm_hidden=64,
num_classes=4):
        super().__init__()
        self.gnn = FrameGNN()
        self.frame_lin = nn.Linear(node_embed, frame_embed)
        self.lstm = nn.LSTM(frame_embed, lstm_hidden, batch_first=True)
        self.classifier = nn.Linear(lstm_hidden, num_classes)
    def forward(self, graph_seq):
        frame_embs = []
        for g in graph_seq:
            x = torch.tensor(g['X'],dtype=torch.float32,device=CFG['device'])
            ei =
torch.tensor(g['edge_index'],dtype=torch.long,device=CFG['device'])
            node_emb = self.gnn(x, ei)
            frame_embs.append(node_emb.mean(dim=0))

```

```

        seq = torch.stack(frame_embs,dim=0).unsqueeze(0)
        seq = F.relu(self.frame_lin(seq))
        lstm_out,_ = self.lstm(seq)
        return self.classifier(lstm_out[:,-1,:])
else:
    GNN_LSTM_Model=None

def train_seq_model(model, seqs, epochs, device):
    opt=torch.optim.Adam(model.parameters(),lr=CFG['lr'])
    crit=nn.CrossEntropyLoss(); model.to(device)
    for ep in range(epochs):
        tot=0
        for seq,lbl in seqs:
            if isinstance(model,LSTMOnly):
                frames=np.stack([np.mean(f['X'],axis=0)[:5] for f in seq])
                x=torch.tensor(frames,dtype=torch.float32,device=device).unsqueeze(0)
            )
            y=torch.tensor([lbl],dtype=torch.long,device=device)
            out=model(x); loss=crit(out,y)
        else:
            out=model(seq);
        y=torch.tensor([lbl],dtype=torch.long,device=CFG['device'])
        loss=crit(out,y)
        opt.zero_grad(); loss.backward(); opt.step(); tot+=loss.item()
    print(f"Epoch {ep+1}/{epochs} loss {tot/len(seqs):.4f}")

def eval_seq_model(model, seqs, device):
    preds,gts=[],[]
    with torch.no_grad():
        for seq,lbl in seqs:
            if isinstance(model,LSTMOnly):
                frames=np.stack([np.mean(f['X'],axis=0)[:5] for f in seq])
                x=torch.tensor(frames,dtype=torch.float32,device=device).unsqueeze(0)
            )
            out=model(x); pred=int(out.argmax(1).cpu())
        else:
            out=model(seq); pred=int(out.argmax().cpu())
        preds.append(pred); gts.append(lbl)
    return accuracy_score(gts,preds), f1_score(gts,preds,average='macro')

def main():
    results=[]
    for track in CFG['tracking_files']:
        base=os.path.basename(track)

```

```

print(f"\n=== Processing {base} ===")
long_df=wide_to_long_tracking(track)
if base in CFG['event_files']:
    ev_file=CFG['event_files'][base]
    active=get_active_frames(ev_file)
    long_df=long_df[long_df['frame_id'].isin(active)]
    print(f"Filtered to {len(long_df)} rows (active play only)")
    long_df=compute_kinematics(long_df)
    players_order=sorted(long_df['player_id'].unique())

for k in [4,5,6]:
    seqs,kmeans,Xs,labels=build_sequences_and_labels(long_df,players_order,n_clusters=k)
    if not seqs: continue
    train,val,test=seqs[:int(0.6*len(seqs))],
seqs[int(0.6*len(seqs)):int(0.8*len(seqs))], seqs[int(0.8*len(seqs)):]

    # LSTM-only
    lstm=LSTMOnly(num_classes=k)
    train_seq_model(lstm,train,epochs=CFG['epochs'],device=CFG['device'])
    acc,f1=eval_seq_model(lstm,test,device=CFG['device'])
    sil=silhouette_score(Xs,labels)
    results.append([base,k,"LSTM-only",acc,f1,sil])

    # GNN+LSTM
    if GNN_LSTM_Model:
        gnn=GNN_LSTM_Model(num_classes=k)
        train_seq_model(gnn,train,epochs=CFG['epochs'],device=CFG['device
'])
        acc,f1=eval_seq_model(gnn,test,device=CFG['device'])
        sil=silhouette_score(Xs,labels)
        results.append([base,k,"GNN+LSTM",acc,f1,sil])

print("\n===== Cluster Sweep Results =====")
print("Dataset Clusters Model Acc F1 Silhouette")
for r in results:
    print(f"{r[0]:12s} {r[1]:3d} {r[2]:10s} {r[3]:.3f} {r[4]:.3f} {r[5]:.3f}")

if __name__=="__main__":
    main()

```



```

import pandas as pd
import matplotlib.pyplot as plt

# Replace this with your actual 'results' variable from the training script
results = [
    ['tracking_home.csv', 4, 'LSTM-only', 0.32, 0.15, 0.12],
    ['tracking_home.csv', 4, 'GNN+LSTM', 0.48, 0.41, 0.33],
    ['tracking_home.csv', 5, 'LSTM-only', 0.36, 0.18, 0.14],
    ['tracking_home.csv', 5, 'GNN+LSTM', 0.52, 0.45, 0.36],
    ['tracking_home.csv', 6, 'LSTM-only', 0.39, 0.22, 0.17],
    ['tracking_home.csv', 6, 'GNN+LSTM', 0.55, 0.49, 0.40],
    # (you can append from other datasets too)
]

df = pd.DataFrame(results, columns=["Dataset", "Clusters", "Model",
    "Accuracy", "F1_macro", "Silhouette"])

# --- Summary table ---
summary = df.groupby("Model")[["Accuracy", "F1_macro",
    "Silhouette"]].mean().round(3)
print("\n=== FINAL SUMMARY TABLE ===")
print(summary)

# --- Plot ---
fig, axes = plt.subplots(1, 3, figsize=(15,5))

metrics = ['Accuracy', 'F1_macro', 'Silhouette']
titles = ['Accuracy vs Clusters', 'F1 Score vs Clusters', 'Silhouette vs Clusters']

for i, metric in enumerate(metrics):
    for model in df['Model'].unique():
        sub = df[df['Model']==model]
        axes[i].plot(sub['Clusters'], sub[metric], marker='o', label=model)
    axes[i].set_title(titles[i])
    axes[i].set_xlabel("Clusters (k)")
    axes[i].set_ylabel(metric)
    axes[i].grid(True)
    axes[i].legend()

plt.suptitle("Model Performance Comparison: LSTM-only vs GNN+LSTM",
    fontsize=14, fontweight='bold')
plt.tight_layout(rect=[0,0,1,0.95])
plt.show()

```

CHAPTER 9

BIBLIOGRAPHY

BIBLIOGRAPHY

- [1] A. Bialkowski, P. Lucey, P. Carr, S. Denman, S. Sridharan, and I. Matthews, "Large-scale analysis of soccer matches using spatiotemporal tracking data," *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pp. 725–730, 2014.
- [2] A. Bialkowski, P. Lucey, P. Carr, Y. Yue, and I. Matthews, "Team formation in sports," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 11, pp. 2138–2149, 2016.
- [3] J. Gudmundsson and M. Horton, "Spatio-temporal analysis of team sports: A survey," *IEEE Transactions on Visualization and Computer Graphics*, vol. 23, no. 1, pp. 370–386, Jan. 2017.
- [4] N. Andrienko, G. Andrienko, G. Budziak, J. Dykes, G. Fuchs, and T. von Landesberger, "Visual analysis of pressure in football," *Data Mining and Knowledge Discovery*, vol. 31, no. 6, pp. 1793–1839, 2017.
- [5] H. M. Le, Y. Yue, P. Carr, and P. Lucey, "Coordinated multi-agent imitation learning," *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 1995–2003, 2017.
- [6] Metrica Sports, "Sample open tracking and event data," Available: <https://github.com/metrica-sports/sample-data>, 2020.
- [7] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2017.
- [8] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," *International Conference on Learning Representations (ICLR)*, 2018.
- [9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [10] Y. Yue, P. Lucey, and P. Carr, "Learning fine-grained spatial models for dynamic sports play prediction," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2642–2651, 2014.
- [11] A. Decroos, J. Bransen, J. Van Haaren, and J. Davis, "Actions speak louder than goals: Valuing player actions in soccer," *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1851–1861, 2019.
- [12] D. Cervone, A. D’Amour, L. Bornn, and K. Goldsberry, "A multiresolution stochastic process model for predicting basketball possession outcomes,"

- Journal of the American Statistical Association*, vol. 111, no. 514, pp. 585–599, 2016.
- [13] A. Spearman, "Beyond expected goals," *OptaPro Analytics Forum*, 2018.
- [14] K. Takeuchi and T. Yamanaka, "Detecting formations in soccer play from tracking data," *Proceedings of the 21st ACM SIGKDD Conference on Knowledge Discovery and Data Mining Workshop on Large-Scale Sports Analytics*, 2014.
- [15] S. Yue and A. Lucey, "Identifying tactical structures in football using spatio-temporal analysis and unsupervised learning," *Journal of Sports Analytics*, vol. 7, no. 3, pp. 201–219, 2021.
- [16] J. Fernandez and D. Bornn, "Wide open spaces: A statistical technique for measuring space creation in professional soccer," *MIT Sloan Sports Analytics Conference*, 2018.
- [17] D. Link, M. Lang, M. Seidenschwarz, and A. Lames, "Real time quantification of dangerousity in football using spatiotemporal tracking data," *PLOS ONE*, vol. 11, no. 12, e0168768, 2016.
- [18] X. Zhang, Z. Li, and J. Wang, "Graph-based spatiotemporal feature learning for sports player trajectory analysis," *IEEE Access*, vol. 8, pp. 221981–221991, 2020.

